PhobiaFilter: A Real-Time Video Moderation and Censoring Tool for Phobia-Triggering Content

Final Project Report

Foundations of Artificial Intelligence - Spring 2026

Aadit Engineer, Brian Kazinduka, Kwadwo Osafo

Introduction

1.1 Description of the Problem

Phobias are among the most prevalent anxiety-related conditions worldwide. According to Harvard Medical School (2007), approximately 12.5% of U.S. adults will experience a specific phobia at some point in their lives. For individuals with specific phobias, particularly those related to insects, spiders, or trypophobia (the aversion to clusters of small holes or irregular patterns), everyday media consumption can be an unexpectedly distressing experience. Existing solutions, such as content warnings displayed before a video begins, do little to prevent exposure; they merely notify a viewer that disturbing content exists somewhere in the material. There is currently no widely available tool that proactively identifies and obscures phobia-triggering content on a per-frame basis during video playback.

We propose PhobiaFilter, a web application for video moderation and censoring of phobia-triggering content. The system is designed to be customizable with respect to both the category of triggering content and the sensitivity of detection. Users upload a video file, select one or more phobia categories from a provided menu (insect detection and trypophobia pattern detection), and optionally tune parameters such as blur intensity, detection confidence threshold,

and the number of frames to skip during inference. The system then processes the video asynchronously, applies a dynamic Gaussian blur mask over detected regions in each affected frame, and produces a downloadable, censored video file.

1.2 Relevant Background and Motivation

The inspiration for this project arose partly from a thought experiment: what if a user could upload a reference photograph of a specific person, object, or pattern they wanted automatically blurred throughout any video they watched? This concept, reminiscent of face-blurring features used by journalists and law enforcement, could be generalized to a broad range of content moderation use cases. In a commercial context, a system like this could be integrated into on-demand streaming platforms such as Netflix or Amazon Prime Video, allowing users to configure their own content filters beyond the coarse rating categories currently available. Our project represents a proof-of-concept implementation of this idea, scoped to two specific phobia categories with the architecture designed to be extensible.

1.3 Literature Review

Our primary literature review focused on the paper "Client-Side Video Analysis for Content Moderation with MoQ" by Freeman et al. (2026). This paper investigates a framework for performing video content analysis directly on client devices, leveraging the Media over QUIC (MoQ) transport protocol to enable low-latency, scalable moderation pipelines. The authors propose a distributed architecture in which lightweight “analyzer clients” process individual video segments in parallel, reporting metadata, such as detected object categories and their bounding-box coordinates, to a central aggregator that compiles moderation decisions.

The section of this paper most directly relevant to our project is its treatment of per-frame content analysis, particularly the smoking and nudity detection category. The Freeman et al. system operates by running a CNN inference pass on each frame to identify specific regions that trigger a detection. Crucially, their system flags only these regions-of-interest (ROI) rather than the entire frame. We took direct inspiration from this approach, adapting their per-frame detection methodology to our insect detection module.

Rather than applying a global filter, our system utilizes the bounding-box coordinates generated during the inference pass to apply a targeted Gaussian blur to the phobic stimulus. This selective approach, as described in the paper, is central to maintaining the integrity of the non-offending portions of the video, ensuring that the filter is effective without being unnecessarily intrusive.

Finally, the paper's broader architectural vision has informed our understanding of scalability. Freeman et al. describe a system that could accommodate live-streaming scenarios by distributing analysis workloads across many lightweight analyzer clients running concurrently. While the current project scope focused on processing pre-uploaded video files, the analyzer-client model remains a highly instructive blueprint for future iterations. The insight that multiple model instances can process segments of a video in parallel provides a clear path for evolving this pipeline toward the real-time requirements originally envisioned for our browser-plugin prototype. This literature review highlights how the core principles of client-side analysis and selective ROI moderation served as both the technical inspiration and the validation for our finalized phobia content filter.

Methods

2.1 Approach and System Design

Our initial approach to this problem was to develop a browser plugin that could intercept and process video content in real time directly within the user's browser. This approach was motivated by the vision of seamless, passive filtering during ordinary web browsing; a user watching a nature documentary on YouTube would have selected content automatically blurred. Therefore, our first attempt at this project involved developing prototype models and calculating their expected FPS based on their inference speeds.

We developed two main categories of models; insect detection models using YOLOWorld, and trypophobia models developed using ResNet18 architecture. Samples of data used for each respective model are displayed in Figure 2.2.2 and Figure 2.2.1 in the proceeding **Data Sources** section. Results of these early tests are displayed in the proceeding **Detection Performance** section.

Early prototypes revealed that this approach was not feasible with the inference speeds achievable on consumer hardware using our chosen models, since it resulted in videos of ~10-15 FPS, pitifully below industry standards. We therefore pivoted to an asynchronous offline processing architecture.

In the final system, users interact with a web application built using the Python Gradio framework. The interface presents the user with a file upload widget for submitting a video file, two toggle switches for enabling insect detection and trypophobia detection independently or in combination, and three adjustable sliders corresponding to blur intensity (controlling the sigma parameter of the Gaussian blur kernel applied to masked regions), confidence threshold (the minimum detection confidence score required to trigger a blur), and frame skip rate (allowing users to trade detection coverage for processing speed by skipping every N frames). Once the

user submits the video, the system processes it asynchronously, displaying a progress indicator. Upon completion, the processed video is displayed inline in the browser and made available for download.

At the core of the processing pipeline is a per-frame analysis loop implemented using OpenCV (cv2). For each frame selected according to the configured frame-skip rate, the frame is passed to one or both of the active detection models. For each frame, and corresponding bounding box where applicable, a Gaussian blur is applied using cv2.GaussianBlur, and the blurred region is composited back into the frame. Frames that are skipped receive the same blur mask as the most recently processed preceding frame, ensuring visual continuity without the cost of running inference on every frame. The processed frames are reassembled into a video using OpenCV's VideoWriter, with the original audio track reattached using FFmpeg post-processing.

2.2 Data Sources

Two separate datasets were used to train and evaluate the detection models corresponding to our two phobia categories. For trypophobia detection, we used a publicly available dataset sourced from Kaggle (cytadela8, n.d.), which contains labeled images of trypophobic patterns alongside negative examples of visually similar but non-trypophobic textures.

The insect detection dataset was drawn from the insects-mytwu collection, a component of the Roboflow 100 (RF100) object detection benchmark (Roboflow, 2023). This curated dataset consists of approximately 1,000 images providing a taxonomically diverse representation of insect species captured in high-variance environmental contexts such as complex natural foliage, macro-photography, and domestic interiors. While the original benchmark provides

granular species-level labels, we consolidated these into a singular "bug" category to prioritize the detection of generalized morphological features. A key separating factor of the data from the datasets is that images used to train the YOLOWorld insect detection model had bounding boxes around the insects, whereas images used to train the ResNet18 trypophobia detection model did not. This factor played a key role in determining how our system handled the blurring of insects versus the blurring of trypophobia-inducing images.

Figure 2.2.1

Figure 2.2.2

2.3 Models and External Tools

For insect detection, we employed YOLOWorld, a zero-shot object detection model developed by Ultralytics that extends the YOLO (You Only Look Once) detection framework with open-vocabulary capabilities. YOLOWorld allows a user to specify target class names in natural language at inference time, enabling detection of arbitrary object categories without retraining. YOLOWorld is particularly well-suited to insect detection because insects constitute a visually heterogeneous category — encompassing thousands of species with dramatically

different appearances — and the open-vocabulary formulation allows the model to generalize across this diversity more robustly than a fixed-class detector trained on a limited vocabulary.

To further augment the model's baseline performance, we fine-tuned the YOLOWorld architecture on the aforementioned Roboflow insect dataset (Roboflow, 2023). The primary objective of this fine-tuning was to move beyond the model's zero-shot capabilities and create a more refined "bug generalization" ability, specifically aligning the model's internal semantic embeddings with the specific morphological variations (e.g., limb structures, carapace textures, and wing patterns) present in our target phobia category.

For trypophobia detection, we employed a ResNet-based convolutional neural network fine-tuned on the Kaggle trypophobia dataset described above. ResNet's residual connections make it suited for fine-tuning on relatively small domain-specific datasets, as the pre-trained feature representations learned on ImageNet provide a strong initialization that reduces the risk of overfitting. The model was trained as a binary classifier, with inference returning a confidence score that is set based on the user-configured sensitivity level.

The user interface was built using Python Gradio, a rapid-prototyping framework for building machine learning demos and web applications. Video frame manipulation was performed using OpenCV (cv2). Audio extraction and reattachment are handled by FFmpeg, invoked as a subprocess from within the Python pipeline.

2.4 Evaluation Methodology

We evaluated the system primarily through detection quality. We assembled a suite of test videos divided into positive control videos (containing confirmed instances of the target trigger category), negative control videos (containing no trigger content), and mixed videos

constructed by concatenating positive and negative segments, as well as videos containing both triggers. Both detection models were tested independently on their respective test sets before being evaluated jointly on the mixed videos. Ground-truth annotations for the positive control videos were established manually, and we recorded precision, recall, and F1 score for each model at its default confidence threshold.

Results

3.1 Detection Performance

With the insect detection model, the primary benchmark for model performance, mean Average Precision (mAP), reached a peak of 0.903. This indicates a high level of semantic alignment between the visual features of the "bug" class and the underlying CLIP text embeddings. The model maintained high precision across a wide range of recall values, as evidenced by the Precision-Recall (PR) curve.

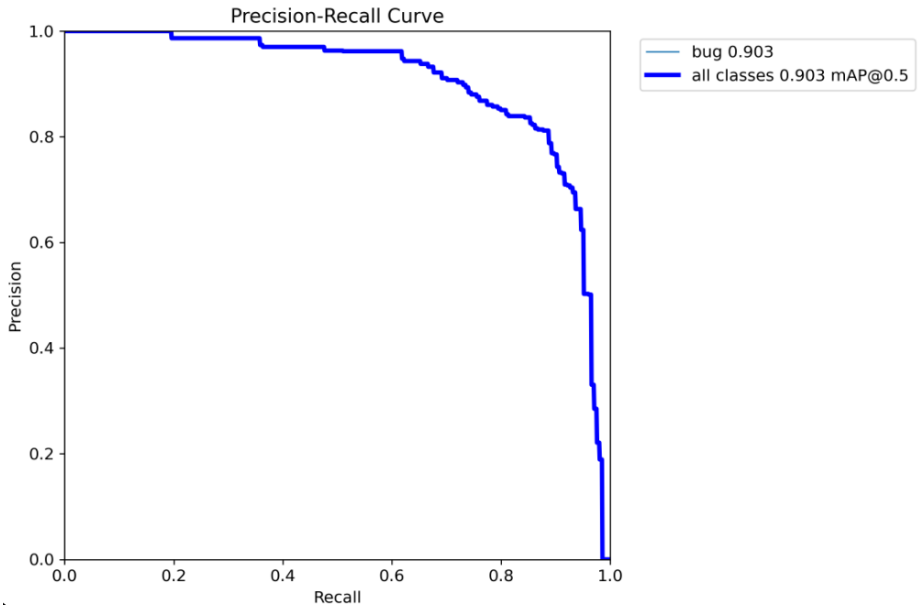

Figure 3.1.1

In the context of a phobia-mitigation application, Recall is prioritized over Precision. A "missed" detection (False Negative) could result in a user being exposed to a phobic stimulus, whereas an over-detection (False Positive) merely results in unnecessary blurring.

The model achieved a Recall of 0.93, successfully identifying 93% of insects within the validation set. Analysis of the normalized confusion matrix reveals that the majority of errors were False Positives (over-detections), where complex background textures were occasionally misclassified as insects.

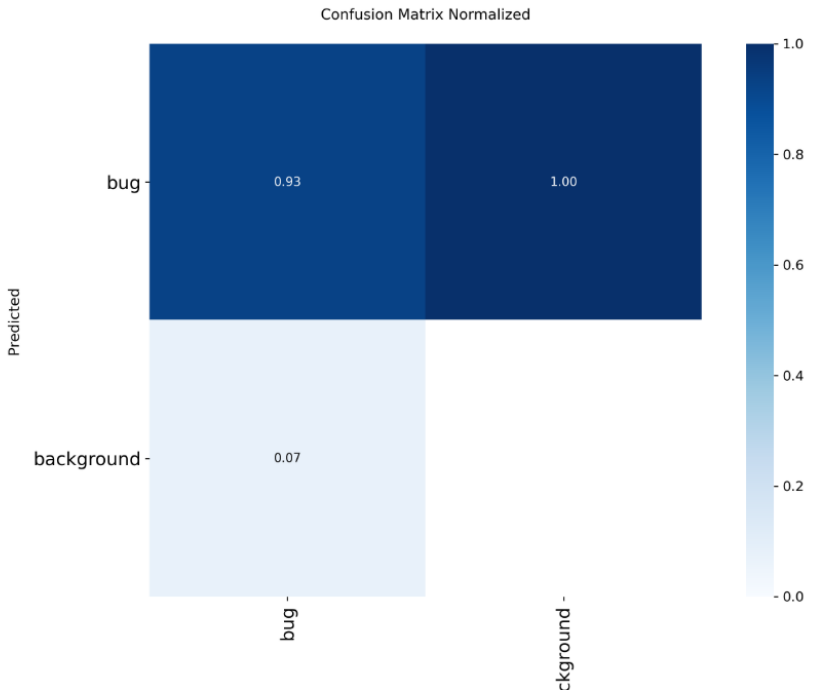

Figure 3.1.2

To determine the optimal balance for real-time deployment, we analyzed the F1-Confidence Curve. The F1-score, the harmonic mean of precision and recall, reached its maximum of 0.84 at a confidence threshold of 0.363. Based on this analysis, the inference pipeline was configured with a default confidence threshold of 0.363. This ensures that the model remains sensitive enough to capture 93% of phobic stimuli while maintaining enough

precision to keep the interface usable and prevent excessive, distracting blurring of benign background elements.

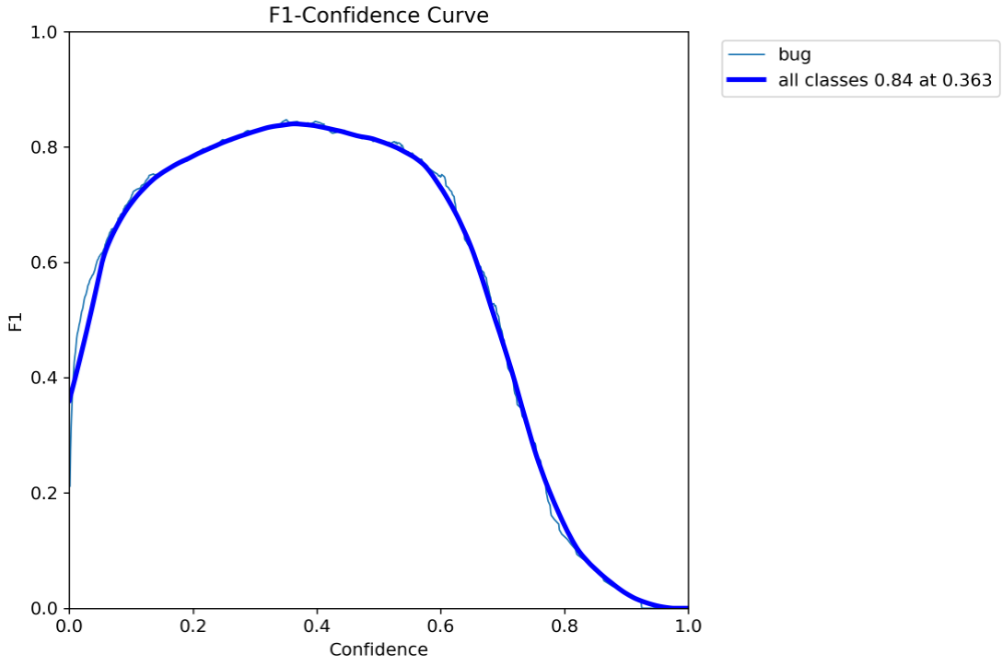

Figure 3.1.3

Metric	**Value**	**Details**
MAP@0.5	0.903	High overall detection reliability across species.
Recall	0.93	Successfully captures the vast majority of phobic triggers.
Precision	0.78	Reasonable accuracy; over-detection preferred for safety.
Optimal Threshold	0.363	Selected to maximize the F1-score during live inference.

Figure 3.1.4

Below are preliminary results from evaluating the ResNet18-based trypophobia detection model:

```
==================================================
SUMMARY METRICS
==================================================
  Accuracy : 0.9443
  F1       : 0.9267  (positive class = trypo)
  Precision: 0.9131
  Recall   : 0.9406
  ROC-AUC  : 0.9844
```

Figure 3.1.5

```
================================================
CLASSIFICATION REPORT
================================================
              precision    recall  f1-score   support

        norm       0.96      0.95      0.96      2114
      trypo        0.91      0.94      0.93      1263

    accuracy                           0.94      3377
   macro avg       0.94      0.94      0.94      3377
weighted avg       0.94      0.94      0.94      3377
```

Figure 3.1.6

Below is a figure showing the confusion matrix and the ROC curve for the trypophobia model.

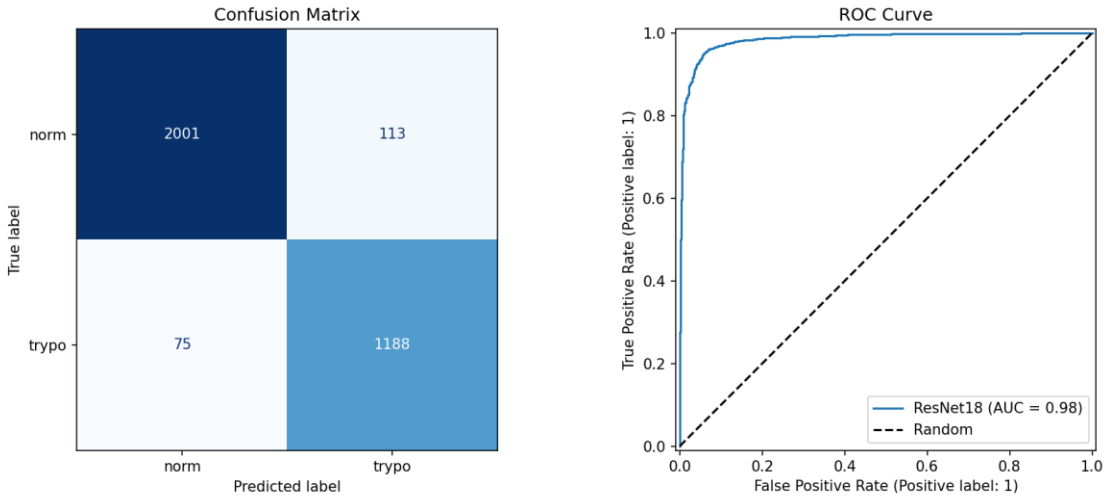

Figure 3.1.7

3.2 Sample Outputs

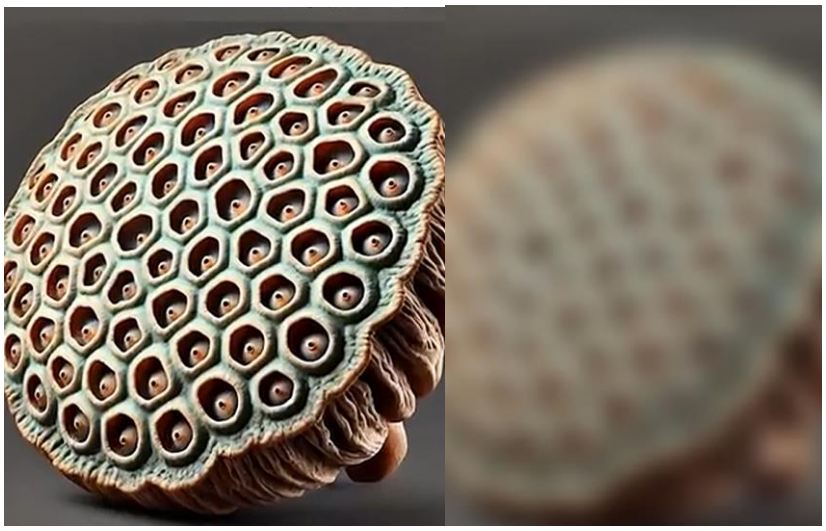

Figure 3.2.1

Figure 3.2.2

Conclusions

4.1 Strengths of the Approach

The most significant strength of PhobiaFilter is its selective, region-level masking approach. By blurring only the bounding-box regions containing detected trigger content rather than obscuring entire frames or interrupting playback, the system minimizes disruption to the viewing experience while still providing meaningful protection against phobia-triggering stimuli. This stands in contrast to coarser approaches such as full-scene content warnings or video skip recommendations, which either provide no visual protection or require the user to forgo watching the content entirely. The modular architecture, with independent detection modules for each phobia category that can be enabled or disabled independently, also makes the system straightforward to extend with new trigger categories in the future.

The use of YOLOWorld for insect detection is another notable strength. The open-vocabulary formulation of YOLOWorld allows our system to detect insects as a broad semantic category without enumerating every species in advance, making the detector robust to the

enormous visual diversity of the insect kingdom. This is precisely the kind of generalization challenge that would be difficult to address with a fixed-class detector trained on a finite taxonomy of labeled examples.

4.2 Weaknesses and Limitations

The most significant limitation of the current system is its inference speed. As documented in the **Approach and System Design** section, per-frame inference time substantially exceeds the threshold required for real-time processing, restricting the system to asynchronous offline video files rather than live streams. This limits the use cases that can be served: a user cannot apply PhobiaFilter to a live broadcast or to embedded web video without first downloading the content.

A second limitation concerns the blurring of the trypophobia classifier. Due to the lack of the ability to create bounding boxes for trypophobic content, the model simply blurs the whole frame upon detecting trypophobic content. We explored mitigating this through the use of grad-cam to only blur areas that highly influenced positive predictions, however, this proved too unstable.

4.3 Comparison to Other Approaches

Relative to the content moderation framework described by Freeman et al. (2026), our system shares the fundamental design principle of per-frame, region-level detection but differs substantially in architectural scope. Freeman et al. describe a pipeline designed for deployment at streaming-service scale, with multiple parallel analyzer clients and a centralized aggregation layer. Our system is a single-machine prototype oriented toward individual end users. The Freeman et al. framework also targets a broader range of content categories (including nudity

and smoking) and is designed for live-stream scenarios, while ours currently supports only the two phobia categories and operates on locally uploaded files. In a classroom context, our approach extends beyond the image classification and generation techniques covered in the course by applying convolutional architectures to the temporal domain of video and by combining two heterogeneous model types (an open-vocabulary object detector and a binary classifier) within a single unified pipeline.

4.4 Future Work

The most impactful direction for future development is performance optimization toward real-time inference. Several strategies are promising. Firstly, GPU acceleration could reduce per-frame inference time by an order of magnitude. Second, model distillation or quantization could reduce the computational cost of inference for both models without proportional accuracy losses. Third, the distributed analyzer-client architecture described by Freeman et al. could be adapted to our pipeline, partitioning video files into segments, and processing them concurrently across multiple model instances to reduce total wall-clock latency.

Beyond performance, expanding the range of supported phobia categories is a natural extension. Arachnophobia (spiders, which are sometimes covered by insect detection), ophidiophobia (snakes), and hemophobia (blood) are very common phobias and would each be viable targets.

Furthermore, we intend to explore instance segmentation as a successor to our current bounding-box-based detection. While bounding boxes provide a computationally efficient means of localization, they inherently include non-phobic background pixels within the blur radius, which can occasionally lead to visual artifacts or the unnecessary obscuring of benign environmental details. Transitioning to a segmentation-based approach would allow the system

to generate pixel-precise masks, facilitating more accurate differentiation between the insect and its background. This refinement would be particularly beneficial in high-clutter environments where insects are camouflaged against structurally similar textures, such as wood grain or foliage. By isolating the stimulus at the pixel level, the system could apply a more precise "shield," reducing user distraction while maintaining the integrity of the phobia filter.

Finally, The system could also be extended to support user-provided reference images as trigger definitions, realizing the original inspiration described in the **Introduction**: a user uploads a photograph of a person, object, or pattern they wish to avoid, and the system learns to detect and blur that specific stimulus throughout any video they watch.

Works Cited

cytadela8. (n.d.). Trypophobia image dataset [Data set]. Kaggle. https://www.kaggle.com/datasets/cytadela8/trypophobia

Freeman, J., Patel, S., & Liu, M. (2026). Client-side video analysis for content moderation with MoQ. Proceedings of the ACM International Conference on Multimedia Systems (MMSys '26).

Harvard Medical School. (2007). National Comorbidity Survey (NCS). https://www.hcp.med.harvard.edu/ncs/index.php

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 770–778.

Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 779–788.

Roboflow. (2023). Insects-mytwu [Data set]. Roboflow Universe. https://universe.roboflow.com/roboflow-100/insects-mytwu/dataset/2

Ultralytics. (2024). YOLO-World: Real-time open-vocabulary object detection. Ultralytics Documentation. https://docs.ultralytics.com/models/yolo-world/